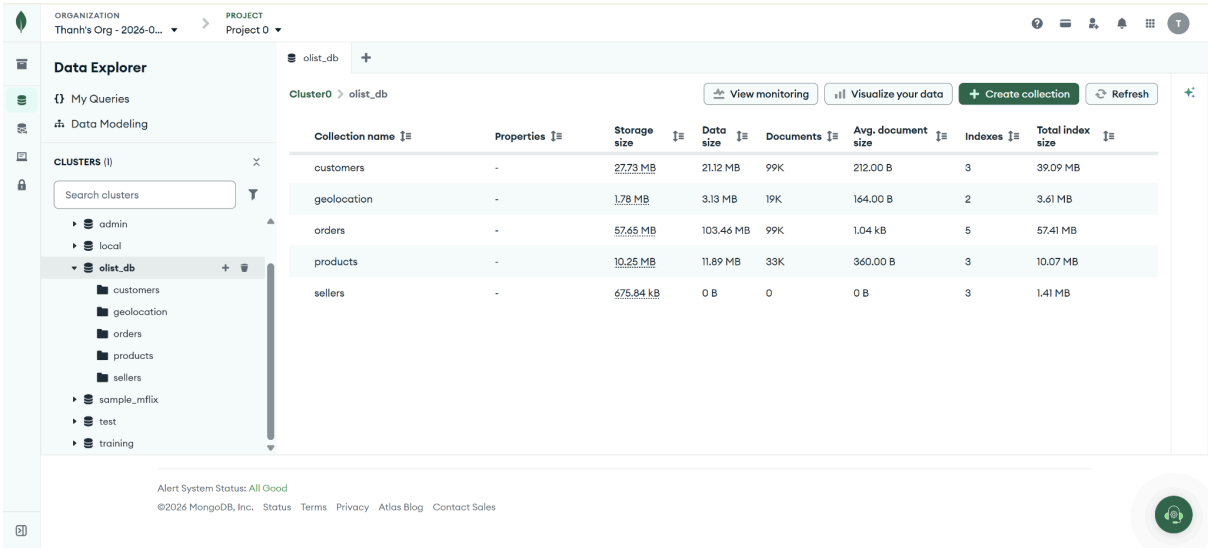
MINH CHỨNG XÂY DỰNG CƠ SỞ DỮ LIỆU MONGODB

Project: Capstone DA_ADV_04
Dataset: Olist Brazilian E-Commerce

1. Xây dựng cơ sở dữ liệu MongoDB

Trong project Capstone DA_ADV_04, bộ dữ liệu Olist Brazilian E-Commerce ban đầu được cung cấp dưới dạng nhiều tệp CSV chứa thông tin về khách hàng, đơn hàng, sản phẩm, người bán, thanh toán, đánh giá và vị trí địa lý. Sau khi dữ liệu được đọc và xử lý bằng Python, các dữ liệu cần thiết được chuyển đổi sang định dạng JSON để phù hợp với mô hình lưu trữ document của MongoDB. Toàn bộ quá trình xử lý và chuyển đổi này được thực hiện trong notebook CSV_to_JSON_MongoDB.ipynb, đồng thời các tệp JSON kết quả cũng được lưu lại trong thư mục bài nộp để làm dữ liệu đầu vào cho MongoDB.

MongoDB Atlas được lựa chọn làm hệ quản trị cơ sở dữ liệu NoSQL cho giai đoạn này của project. Python kết nối đến MongoDB Atlas thông qua thư viện PyMongo, sau đó các document JSON được đưa vào database **olist_db**. Kết quả sau quá trình import là database được tổ chức thành 5 collections gồm **customers, products, sellers, geolocation** và **orders**. Mỗi collection đại diện cho một nhóm thông tin chính trong bộ dữ liệu Olist và được sử dụng làm nguồn dữ liệu cho các bước tiếp theo của pipeline.

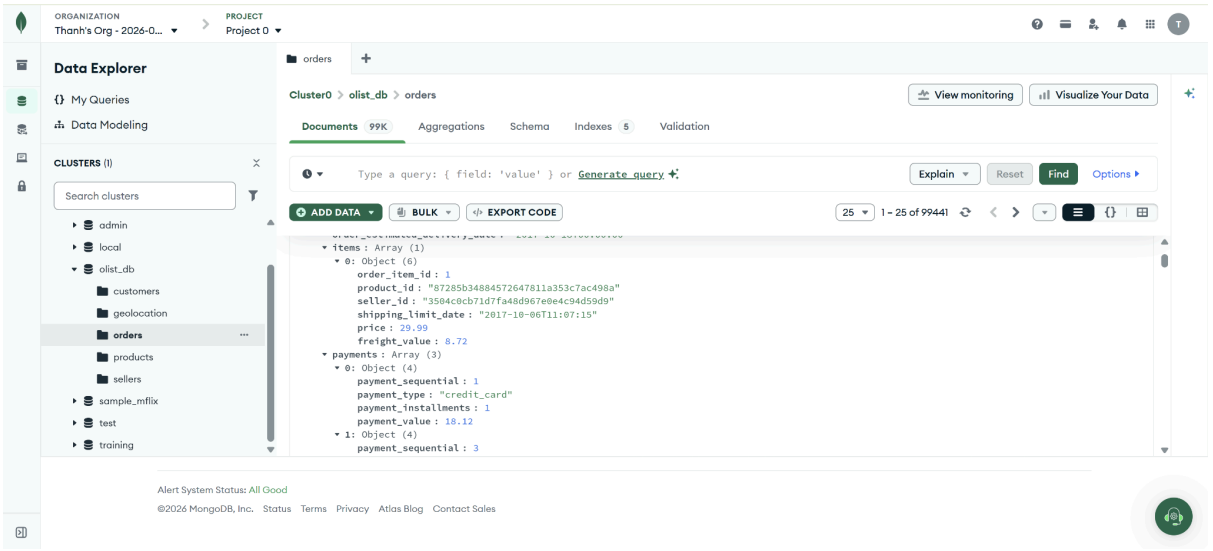


Hình 1. Database olist_db và các collections được tạo trên MongoDB Atlas.

2. Tổ chức và kiểm tra dữ liệu trên MongoDB

Sau khi quá trình import hoàn tất, dữ liệu trong các collections được kiểm tra trực tiếp trên MongoDB Atlas nhằm xác nhận các document đã được lưu trữ thành công và có cấu trúc phù hợp. Trong đó, collection **orders** là collection quan trọng nhất vì ngoài các thông tin cơ bản của đơn hàng như mã đơn hàng, khách hàng, trạng thái và các mốc thời gian, dữ liệu liên quan đến từng đơn hàng còn được tổ chức dưới dạng nested data.

Cụ thể, các thông tin về sản phẩm trong đơn hàng (items), phương thức và giá trị thanh toán (payments) cũng như thông tin đánh giá (reviews) được gắn với document của đơn hàng tương ứng. Cách tổ chức này tận dụng đặc điểm của mô hình document trong MongoDB, cho phép những dữ liệu có quan hệ chặt chẽ với một đơn hàng được lưu trữ cùng nhau thay vì phải duy trì hoàn toàn dưới dạng các bảng riêng biệt như trong cơ sở dữ liệu quan hệ.



Hình 2. Cấu trúc document trong collection orders sau khi dữ liệu được import.

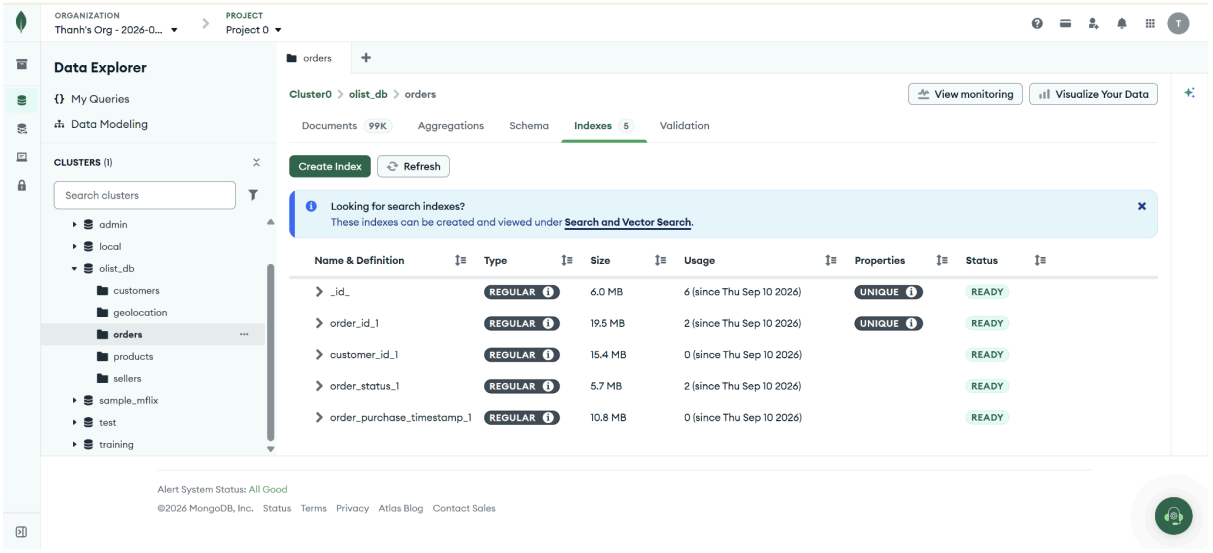
3. Tạo Index trên MongoDB

Sau khi dữ liệu được đưa vào MongoDB, các index được tạo trên một số trường định danh quan trọng như *customer_id*, *product_id*, *seller_id* và *order_id*. Việc tạo index không làm thay đổi nội dung của các document mà nhằm hỗ trợ MongoDB truy xuất dữ liệu hiệu quả hơn. Khi một trường thường xuyên được sử dụng để tìm kiếm hoặc xác định document, index cho phép MongoDB sử dụng cấu trúc tra cứu thay vì phải lần lượt quét toàn bộ document trong collection.

Bên cạnh việc hỗ trợ hiệu năng truy vấn, một số trường định danh được thiết lập dưới dạng unique index nhằm duy trì tính duy nhất của dữ liệu. Các trường như customer_id, product_id hay order_id đóng vai trò nhận diện các đối tượng trong bộ dữ liệu, do đó việc hạn chế nhiều document có cùng một giá trị định danh giúp giảm nguy cơ phát sinh dữ liệu trùng lặp trong quá trình import hoặc xử lý lại dữ liệu.

Việc tạo index cũng có ý nghĩa đối với pipeline tổng thể của project. MongoDB không phải là điểm kết thúc của dữ liệu mà tiếp tục đóng vai trò nguồn dữ liệu cho quá trình ETL bằng Python sang SQL Server. Vì vậy, việc tổ chức dữ liệu với các trường định danh rõ ràng, hạn chế trùng lặp và hỗ trợ truy

xuất hiệu quả giúp dữ liệu có trạng thái phù hợp hơn trước khi được đọc và chuyển đổi sang mô hình cơ sở dữ liệu quan hệ ở giai đoạn tiếp theo.



Hình 3. Index được tạo trên trường định danh của collection trong MongoDB Atlas.

4. Kết quả

Sau khi hoàn thành giai đoạn MongoDB, dữ liệu Olist đã được chuyển đổi từ dữ liệu CSV ban đầu sang định dạng JSON và lưu trữ thành công trên MongoDB Atlas. Database **olist_db** gồm 5 collections chính, trong đó dữ liệu đơn hàng được tổ chức theo cấu trúc document có chứa các thông tin liên quan dưới dạng nested data. Các index trên những trường định danh quan trọng cũng được thiết lập nhằm hỗ trợ truy vấn và hạn chế dữ liệu trùng lặp.

Kết quả của giai đoạn này tạo thành nguồn dữ liệu NoSQL hoàn chỉnh cho bước tiếp theo trong pipeline của project. Dữ liệu từ MongoDB Atlas sau đó được Python kết nối, trích xuất và chuyển đổi để đưa sang SQL Server, trước khi tiếp tục được sử dụng cho quá trình phân tích và trực quan hóa dữ liệu.

CSV → JSON → MongoDB Atlas → Python ETL → SQL Server